Interview

Preparation

Complete A-Z Guide for SDE, Android, Web & All Tech Roles

Creator & Author

Mohammad Faiz

Year 2026

Table of Contents

Part 1: Interview Fundamentals & Strategy 4

Chapter 1: Understanding the Interview Process .....

Chapter 2: Interview Types & Formats .....

Chapter 3: Pre-Interview Preparation .....

Chapter 4: During the Interview .....

Chapter 5: Post-Interview Strategy .....

Part 2: Data Structures & Algorithms 11

Chapter 6: Arrays & Strings .....

Chapter 7: Linked Lists .....

Chapter 8: Stacks & Queues .....

Chapter 9: Trees & Graphs .....

Chapter 10: Dynamic Programming .....

Chapter 11: Sorting & Searching .....

Part 3: System Design Interviews 30

Chapter 12: System Design Fundamentals .....

Chapter 13: Scalability & Performance .....

Chapter 14: Database Design .....

Chapter 15: Microservices Architecture .....

Chapter 16: System Design Case Studies .....

Part 4: Android Development Interviews 36

Chapter 17: Android Fundamentals .....

Chapter 18: Kotlin for Android .....

Chapter 19: Android Architecture Components .....

Chapter 20: Android Performance & Optimization .....

Chapter 21: Android Interview Questions .....

Part 5: Web Development Interviews 45

Chapter 22: Frontend Fundamentals .....

Chapter 23: JavaScript Deep Dive.....
Chapter 24: React & Modern Frameworks.....
Chapter 25: Backend Development.....
Chapter 26: Full Stack Interview Questions.....

Part 6: Behavioral & HR Interviews.....56

Chapter 27: Understanding Behavioral Interviews.....
Chapter 28: STAR Method Mastery.....
Chapter 29: Common Behavioral Questions.....
Chapter 30: HR Interview Strategies.....

Part 7: Resume & Portfolio Building.....61

Chapter 31: Creating an ATS-Friendly Resume.....
Chapter 32: Building Your Portfolio.....
Chapter 33: GitHub & Online Presence.....
Chapter 34: Cover Letters & Applications.....

Part 8: Company-Specific Preparation.....67

Chapter 35: FAANG Companies.....
Chapter 36: Startups vs Big Tech.....
Chapter 37: Service-Based Companies.....

Part 9: Mock Interviews & Practice.....71

Chapter 38: Practice Problems by Topic.....
Chapter 39: Mock Interview Scenarios.....
Chapter 40: Final Preparation Checklist.....

Part 1: Interview Fundamentals & Strategy

Welcome to your comprehensive guide to interview preparation! This part will help you understand the interview landscape, develop effective strategies, and build the confidence you need to succeed in any technical interview. Whether you are preparing for your first job or looking to advance your career, this section provides the foundation for interview success.

Chapter 1: Understanding the Interview Process

1.1 The Interview Journey

The technical interview process typically follows a structured journey. Understanding each stage helps you prepare effectively and reduces anxiety.

Stage	Purpose	What to Expect
1. Application	Submit your resume and application	Online form, referral, or direct application
2. Resume Screening	Recruiter reviews your qualifications	ATS scan, manual review
3. Phone/Video Screen	Initial conversation with recruiter	Basic questions, salary discussion
4. Technical Screen	Assess technical skills	Coding challenge, technical discussion
5. On-site/Virtual Onsite	In-depth technical evaluation	Multiple rounds, 3-6 hours
6. Behavioral	Assess culture fit	STAR method questions
7. Offer	Present job offer	Salary negotiation, benefits

1.2 Types of Technical Interviews

1. Coding Interviews
You are given a problem to solve by writing code. These test your problem-solving skills, data structure knowledge, and ability to write clean, efficient code.

2. System Design Interviews
You are asked to design a large-scale system like Twitter, Uber, or a URL shortener. These evaluate your ability to architect complex systems.

3. Behavioral Interviews

Questions about your past experiences, how you handled situations, and your soft skills. These assess culture fit and communication abilities.

4. Domain-Specific Interviews

Questions specific to your field like Android development, web development, machine learning, etc.

1.3 What Interviewers Look For

- Problem-Solving Skills: Can you break down complex problems?
- Technical Knowledge: Do you understand the fundamentals?
- Communication: Can you explain your thought process clearly?
- Code Quality: Is your code clean, readable, and maintainable?
- Collaboration: Can you work with hints and feedback?
- Learning Ability: Can you adapt to new information?

1.4 The Interview Mindset

Success in interviews requires the right mindset. Remember these key principles:

0. **Interviews are Conversations**: Not interrogations. Engage with your interviewer.
1. **Think Out Loud**: Share your thought process. Silent thinking helps no one.
2. **It is Okay to Not Know**: Admitting gaps and learning shows maturity.
3. **Practice Makes Perfect**: The more you practice, the more confident you become.

Chapter 2: Interview Types & Formats

2.1 Phone/Video Screening

This is typically the first technical interaction. It lasts 30-45 minutes and focuses on:

- Basic technical questions
- Simple coding problems (on shared document or IDE)
- Your background and experience
- Your interest in the company

2.2 Take-Home Assignments

Some companies give you a project to complete at home within a deadline.

Tips for Take-Homes:

- Read requirements carefully
- Focus on clean code and documentation
- Include a README with setup instructions
- Write tests if possible
- Do not spend too much time - quality over quantity

2.3 Pair Programming Interviews

You code alongside an interviewer. This format tests:

- Your ability to collaborate
- How you handle feedback
- Your coding speed and style

2.4 Whiteboard Interviews

Traditional format where you solve problems on a whiteboard (physical or virtual).

Whiteboard Tips:

- Start with a high-level approach
- Write legibly and organize your space
- Talk through your solution
- Test your solution with examples

2.5 Online Assessment (OA)

Automated coding tests, often used as a first filter.

OA Platforms:

Platform	Used By

HackerRank	Many companies
LeetCode	Practice and some companies
Codility	Enterprise companies
CodeSignal	Tech companies
HackerEarth	Startups and mid-size

Chapter 3: Pre-Interview Preparation

3.1 Research the Company

Thorough company research shows genuine interest and helps you ask intelligent questions.

- Company products/services and target market
- Recent news, funding rounds, acquisitions
- Company culture and values
- Tech stack and engineering blog
- Interviewers' backgrounds (LinkedIn)

3.2 Study the Job Description

The job description is your study guide. Identify:

- Required skills - ensure you can demonstrate these
- Nice-to-have skills - mention if you have them
- Key responsibilities - prepare relevant examples

3.3 Technical Preparation Timeline

Timeframe	Focus Area
3+ Months Before	DSA fundamentals, language proficiency
2 Months Before	Practice problems, build projects
1 Month Before	Mock interviews, system design study

1 Week Before	Company research, review your projects
Day Before	Rest, light review, prepare questions

3.4 The Day Before the Interview

- Get a good night's sleep
- Prepare your outfit (business casual for most tech companies)
- Test your tech setup (camera, microphone, internet)
- Prepare questions to ask the interviewer
- Plan your route (or quiet space for virtual interviews)
- Do light review - do not cram

Chapter 4: During the Interview

%0%.1 The First 5 Minutes

First impressions matter. Start strong with:

- Arrive 5-10 minutes early (or join virtual call on time)
- Greet warmly and maintain eye contact
- Have a brief introduction ready (elevator pitch)
- Show enthusiasm for the opportunity

%0%.2 Problem-Solving Framework

The UMPIRE Method:

Step	Action
U - Understand	Clarify the problem, ask questions
M - Match	Match to known patterns/algorithms
P - Plan	Outline your approach before coding
I - Implement	Write clean, working code

R - Review	Test with examples, check edge cases
E - Evaluate	Discuss time/space complexity

%0%.3 Communication Tips

- Think out loud - verbalize your thought process
- Listen carefully to hints and feedback
- Ask clarifying questions
- Acknowledge when you are stuck and ask for help
- Summarize your approach before coding

%0%.4 Red Flags to Avoid

- Jumping into code without understanding the problem
- Ignoring hints from the interviewer
- Being defensive about feedback
- Staying silent for long periods
- Not testing your solution

Chapter 5: Post-Interview Strategy

5.1 The Thank You Email

Send a thank you email within 24 hours to each interviewer.

```
Subject: Thank you - [Position] Interview

Hi [Interviewer's Name],

Thank you for taking the time to speak with me about the [Position]
role at [Company]. I enjoyed learning more about [specific topic
discussed] and am excited about the opportunity to contribute to
[Company's] mission.
```

I was particularly interested in [specific detail about the role/company]. My experience with [relevant skill] would allow me to make an immediate impact on your team.

Please let me know if you need any additional information. I look forward to hearing from you.

Best regards,
[Your Name]

5.2 Handling Rejection

Rejection is part of the process. Use it as a learning opportunity:

- Ask for feedback (politely)
- Reflect on what you could improve
- Do not take it personally - many factors are at play
- Apply what you learned to the next interview

5.3 Negotiating Offers

When you receive an offer:

- Express gratitude and enthusiasm
- Ask for time to consider (2-3 days is reasonable)
- Research market rates for your role and location
- Consider the total package (salary, benefits, equity, growth)
- Negotiate respectfully with data to back your request

Part 2: Data Structures & Algorithms

Data Structures and Algorithms (DSA) form the backbone of technical interviews. This part covers the essential topics you need to master, with explanations, examples, and practice problems.

Chapter 6: Arrays & Strings

6.1 Array Fundamentals

Arrays are the most basic and commonly used data structure. They store elements in contiguous memory locations, allowing O(1) access by index.

Time Complexities:

Operation	Time Complexity
Access by index	O(1)
Search (unsorted)	O(n)
Search (sorted)	O(log n) - binary search
Insert/Delete at end	O(1) amortized
Insert/Delete at beginning/middle	O(n)

6.2 Common Array Patterns

Pattern 1: Two Pointers

Used for searching pairs, reversing, or partitioning arrays.

```
// Two Sum - Sorted Array
public int[] twoSum(int[] nums, int target)
  { int left = 0, right = nums.length - 1;

  while (left < right) {
    int sum = nums[left] +
    nums[right]; if (sum == target) {
      return new int[]{left, right};
```

```
        } else if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return new int[]{-1, -1};
}
```

Pattern 2: Sliding Window

Used for subarray/substring problems.

```
// Maximum Sum Subarray of Size K
public int maxSum(int[] nums, int k) {
    int maxSum = 0, windowSum = 0;

    for (int i = 0; i < nums.length; i++)
        { windowSum += nums[i];

        if (i >= k - 1) {
            maxSum = Math.max(maxSum, windowSum);
            windowSum -= nums[i - k + 1];
        }
    }
    return maxSum;
}
```

Pattern 3: Prefix Sum

```
// Range Sum Query
public int rangeSum(int[] nums, int left, int right)
    { int[] prefix = new int[nums.length + 1];
```

```
    for (int i = 0; i < nums.length; i++)
        { prefix[i + 1] = prefix[i] + nums[i];
    }

    return prefix[right + 1] - prefix[left];
}
```

6.3 String Manipulation

Common String Operations:

```
// Check if string is palindrome
public boolean isPalindrome(String s)
    { int left = 0, right = s.length() - 1;

    while (left < right) {
        if (s.charAt(left) != s.charAt(right))
            { return false;
        }
        left++;
        right--;
    }
    return true;
}

// Reverse a string
public String reverse(String s) {
    return new StringBuilder(s).reverse().toString();
}

// Anagram check
```

```
public boolean isAnagram(String s, String t)
  { if (s.length() != t.length()) return false;

  int[] count = new int[26];
  for (int i = 0; i < s.length(); i++)
    { count[s.charAt(i) - 'a']++;
    count[t.charAt(i) - 'a']--;
  }

  for (int c : count) {
    if (c != 0) return false;
  }
  return true;
}
```

6.4 Practice Problems - Arrays

Problem	Difficulty	Pattern
Two Sum	Easy	Hash Map / Two Pointers
Best Time to Buy/Sell Stock	Easy	One Pass
Contains Duplicate	Easy	Hash Set
Product of Array Except Self	Medium	Prefix/Suffix
Maximum Subarray (Kadane)	Medium	Dynamic Programming
3Sum	Medium	Two Pointers
Merge Intervals	Medium	Sorting
Trapping Rain Water	Hard	Two Pointers

Chapter 7: Linked Lists

7.1 Linked List Basics

A linked list is a linear data structure where elements are stored in nodes, and each node points to the next node.

```
// Node
definition class
ListNode {
    int val;
    ListNode next;

    ListNode(int val) {
        this.val = val;
    }
}
```

Operation	Array	Linked List
Access	O(1)	O(n)
Insert at beginning	O(n)	O(1)
Insert at end	O(1) amortized	O(n) or O(1) with tail
Delete	O(n)	O(1) with pointer
Memory	Contiguous	Scattered

7.2 Essential Linked List Operations

```
// Reverse a linked list
public ListNode reverse(ListNode head)
    { ListNode prev = null;
    ListNode current = head;

    while (current != null) {
        ListNode next = current.next;
```

```
        current.next = prev;
        prev = current;
        current = next;
    }
    return prev;
}

// Find middle of linked list
public ListNode findMiddle(ListNode head)
    { ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null)
        { slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

// Detect cycle (Floyd's Algorithm)
public boolean hasCycle(ListNode head)
    { ListNode slow = head;
    ListNode fast = head;

    while (fast != null && fast.next != null)
        { slow = slow.next;
        fast = fast.next.next;

        if (slow == fast) return true;
    }
    return false;
```

```
}
```

7.3 Practice Problems - Linked Lists

Problem	Difficulty
Reverse Linked List	Easy
Merge Two Sorted Lists	Easy
Linked List Cycle	Easy
Middle of Linked List	Easy
Remove Nth Node from End	Medium
Add Two Numbers	Medium
Copy List with Random Pointer	Medium
Merge K Sorted Lists	Hard

Chapter 8: Stacks & Queues

8.1 Stack - LIFO

Stack follows Last In First Out (LIFO) principle. Think of a stack of plates.

```
// Stack using ArrayDeque (preferred over Stack class)
Deque<Integer> stack = new ArrayDeque<>();

stack.push(1); // Add to top
stack.push(2);
int top = stack.peek(); // View top (2)
int popped = stack.pop(); // Remove top
(2) boolean isEmpty = stack.isEmpty();
```

Common Stack Problems:

```
// Valid Parentheses
```

```
public boolean isValid(String s) {
    Deque<Character> stack = new
    ArrayDeque<>();

    for (char c : s.toCharArray()) {
        if (c == '(') stack.push(')');
        else if (c == '{') stack.push('}');
        else if (c == '[') stack.push(']');
        else if (stack.isEmpty() || stack.pop() != c) {
            return false;
        }
    }
    return stack.isEmpty();
}

// Next Greater Element
public int[] nextGreaterElement(int[] nums) {
    int[] result = new int[nums.length];
    Arrays.fill(result, -1);
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < nums.length; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i])
            { result[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return result;
}
```

8.2 Queue - FIFO

Queue follows First In First Out (FIFO) principle. Think of a line at a ticket counter.

```
// Queue using ArrayDeque
Deque<Integer> queue = new ArrayDeque<>();

queue.offer(1); // Add to back
queue.offer(2);
int front = queue.peek(); // View front (1)
int removed = queue.poll(); // Remove front (1)
```

8.3 Practice Problems - Stacks & Queues

Problem	Difficulty
Valid Parentheses	Easy
Min Stack	Easy
Implement Queue using Stacks	Easy
Daily Temperatures	Medium
Evaluate Reverse Polish Notation	Medium
LRU Cache	Medium
Largest Rectangle in Histogram	Hard

Chapter 9: Trees & Graphs

9.1 Binary Tree Basics

```
// Tree Node
definition class
TreeNode {
  int val;
  TreeNode left;
```

```
    TreeNode(int val) {
        this.val = val;
    }
}
```

9.2 Tree Traversals

```
// Inorder Traversal (Left -> Root -> Right)
public void inorder(TreeNode root) {
    if (root == null) return;
    inorder(root.left);
    System.out.print(root.val + " ");
    inorder(root.right);
}

// Preorder Traversal (Root -> Left ->
Right) public void preorder(TreeNode root)
{
    if (root == null) return;
    System.out.print(root.val + " ");
    preorder(root.left);
    preorder(root.right);
}

// Postorder Traversal (Left -> Right ->
Root) public void postorder(TreeNode root)
{
    if (root == null) return;
    postorder(root.left);
    postorder(root.right);
    System.out.print(root.val + " ");
}
```

```
// Level Order Traversal (BFS)
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < levelSize; i++)
            { TreeNode node = queue.poll();
            level.add(node.val);

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

9.3 Binary Search Tree (BST)

BST property: Left subtree < Root < Right subtree

```
// Validate BST
public boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
```

```
}

private boolean validate(TreeNode node, long min, long max)
  { if (node == null) return true;

  if (node.val <= min || node.val >= max) return false;

  return validate(node.left, min, node.val) &&
        validate(node.right, node.val, max);
}

// Search in BST
public TreeNode searchBST(TreeNode root, int val)
  { if (root == null || root.val == val) return root;

  if (val < root.val) {
      return searchBST(root.left, val);
  }
   return searchBST(root.right, val);
}
```

9.4 Graph Basics

Graph representation and traversal algorithms.

```
// Graph representation using adjacency list
class Graph {
  private Map<Integer, List<Integer>> adj;

  public Graph() {
     adj = new HashMap<>();
  }
```

```
    public void addEdge(int u, int v) {
        adj.computeIfAbsent(u, k -> new
        ArrayList<>()).add(v); adj.computeIfAbsent(v, k ->
        new ArrayList<>()).add(u);
    }
}

// BFS traversal
public void bfs(Map<Integer, List<Integer>> graph, int start)
    { Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();

    queue.offer(start);
    visited.add(start);

    while (!queue.isEmpty())
        { int node = queue.poll();
        System.out.print(node + " ");

        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.offer(neighbor);
            }
        }
    }
}

// DFS traversal
public void dfs(Map<Integer, List<Integer>> graph, int start, Set<Integer> visited) {
    visited.add(start);
```

```
        System.out.print(start + " ");

        for (int neighbor : graph.getOrDefault(start, new ArrayList<>())) {
            if (!visited.contains(neighbor)) {
                dfs(graph, neighbor, visited);
            }
        }
    }
```

9.5 Practice Problems - Trees & Graphs

Problem	Difficulty
Maximum Depth of Binary Tree	Easy
Same Tree	Easy
Invert Binary Tree	Easy
Subtree of Another Tree	Easy
Lowest Common Ancestor	Medium
Binary Tree Level Order	Medium
Number of Islands	Medium
Course Schedule (Topological Sort)	Medium
Word Ladder	Hard

Chapter 10: Dynamic Programming

10.1 What is Dynamic Programming?

Dynamic Programming (DP) is a technique for solving complex problems by breaking them down into simpler subproblems, solving each subproblem only once, and storing their solutions.

Key Characteristics:

- Optimal Substructure: Optimal solution can be constructed from optimal solutions to

subproblems

- Overlapping Subproblems: Same subproblems are solved multiple times

10.2 DP Approaches

1. Top-Down (Memoization)

```
// Fibonacci with Memoization
public int fib(int n) {
    int[] memo = new int[n + 1];
    return  fibHelper(n,  memo);
}

private int fibHelper(int n, int[] memo)
    { if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];

    memo[n] = fibHelper(n - 1, memo) + fibHelper(n - 2, memo);
    return memo[n];
}
```

2. Bottom-Up (Tabulation)

```
// Fibonacci with Tabulation
public int fib(int n) {
    if (n <= 1) return n;

    int[] dp = new int[n + 1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
```

```
    }
    return dp[n];
}
```

10.3 Common DP Patterns

Pattern	Example Problems
0/1 Knapsack	Subset Sum, Partition Equal Subset
Unbounded Knapsack	Coin Change, Rod Cutting
Longest Common Subsequence	Edit Distance, Wildcard Matching
Palindromes	Longest Palindromic Substring
Fibonacci Style	Climbing Stairs, House Robber
Grid Traversal	Unique Paths, Minimum Path Sum

10.4 Practice Problems - DP

Problem	Difficulty
Climbing Stairs	Easy
House Robber	Easy
Best Time to Buy/Sell Stock	Easy
Coin Change	Medium
Longest Increasing Subsequence	Medium
Edit Distance	Medium
0/1 Knapsack	Medium
Regular Expression Matching	Hard

Chapter 11: Sorting & Searching

11.1 Essential Sorting Algorithms

```
// Quick Sort
public void quickSort(int[] arr, int low, int high)
  { if (low < high) {
    int pi = partition(arr, low,
    high); quickSort(arr, low, pi -
    1); quickSort(arr, pi + 1, high);
  }
}

private int partition(int[] arr, int low, int high) {
  int pivot = arr[high];
  int i = low - 1;

  for (int j = low; j < high; j+
    +) { if (arr[j] < pivot) {
      i++;
      swap(arr, i, j);
    }
  }
  swap(arr, i + 1,
  high); return i + 1;
}

// Merge Sort
public void mergeSort(int[] arr, int left, int right) {
  if (left < right) {
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
```

```
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

Algorithm	Time (Avg)	Time (Worst)	Space	Stable
Quick Sort	O(n log n)	O(n^2)	O(log n)	No
Merge Sort	O(n log n)	O(n log n)	O(n)	Yes
Heap Sort	O(n log n)	O(n log n)	O(1)	No
Insertion Sort	O(n^2)	O(n^2)	O(1)	Yes
Counting Sort	O(n + k)	O(n + k)	O(k)	Yes

11.2 Binary Search

Binary search is a fundamental algorithm for searching in sorted arrays with O(log n) time complexity.

```
// Iterative Binary Search
public int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid +
        1; else right = mid - 1;
    }
    return -1;
}

// Binary Search Template for Variations
```

```
public int binarySearchTemplate(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (condition(mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }
    return left;
}
```

11.3 Practice Problems - Sorting & Searching

Problem	Difficulty
Binary Search	Easy
Search Insert Position	Easy
First Bad Version	Easy
Find Minimum in Rotated Sorted Array	Medium
Search in Rotated Sorted Array	Medium
Find Peak Element	Medium
Median of Two Sorted Arrays	Hard

Part 3: System Design Interviews

System design interviews assess your ability to design scalable, reliable, and efficient systems. This part covers the fundamental concepts, patterns, and case studies you need to succeed.

Chapter 12: System Design Fundamentals

12.1 What is System Design?

System design is the process of defining the architecture, components, interfaces, and data for a system to satisfy specified requirements. In interviews, you are typically asked to design a large-scale distributed system.

12.2 The System Design Interview Process

Phase	Time	What To Do
Requirements Gathering	2-3 min	Ask clarifying questions
Back-of-Envelope Estimation	2-3 min	Calculate scale and capacity
High-Level Design	10-15 min	Draw system architecture
Deep Dive	15-20 min	Discuss components in detail
Trade-offs & Extensions	5-10 min	Discuss alternatives and future scaling

12.3 Key Concepts to Know

- Horizontal vs Vertical Scaling
- Load Balancing
- Caching (CDN, Application, Database)
- Database Sharding and Replication
- CAP Theorem
- Microservices vs Monolith
- Message Queues

- Rate Limiting

12.4 The 4S Framework

Scenario: Understand the requirements and constraints

Sketch: Draw the high-level design

Substantiate: Deep dive into components

Scale: Discuss bottlenecks and improvements

Chapter 13: Scalability & Performance

13.1 Scaling Strategies

Vertical Scaling (Scale Up):

Add more power (CPU, RAM) to existing machines. Limited by hardware maximums.

Horizontal Scaling (Scale Out):

Add more machines to distribute load. More flexible but adds complexity.

13.2 Load Balancing

Load balancers distribute incoming traffic across multiple servers.

Algorithm	Description
Round Robin	Distribute sequentially to each server
Least Connections	Send to server with fewest active connections
IP Hash	Route based on client's IP address
Weighted	Assign different weights to servers

13.3 Caching Strategies

Cache Levels:

- Client Cache: Browser cache, mobile app cache
- CDN Cache: Cache static content geographically distributed

- Application Cache: Redis, Memcached
- Database Cache: Query cache, buffer pool

Cache Patterns:

Pattern	Description
Cache Aside	Application manages cache
Read Through	Cache reads from DB on miss
Write Through	Write to cache and DB together
Write Behind	Async write to DB

13.4 Database Scaling

Replication:

- Master-Slave: Writes to master, reads from slaves
- Master-Master: Both handle reads and writes

Sharding:

- Horizontal Partition: Split data across multiple databases
- Shard Key: Determine which shard stores the data

Chapter 14: Database Design

14.1 SQL vs NoSQL

Aspect	SQL	NoSQL
Schema	Fixed schema	Flexible schema
Scaling	Vertical	Horizontal
Transactions	ACID compliant	Eventual consistency
Use Case	Complex queries, relationships	High write throughput, unstructured
Examples	MySQL, PostgreSQL	MongoDB, Cassandra, Redis

14.2 Database Indexing

Indexes speed up read operations but slow down writes.

- B-Tree Index: Default, good for range queries
- Hash Index: Good for equality lookups
- Composite Index: Multiple columns
- Covering Index: Index contains all queried columns

14.3 CAP Theorem

In distributed systems, you can only guarantee two of three:

- Consistency: All nodes see the same data
- Availability: Every request gets a response
- Partition Tolerance: System works despite network failures

Chapter 15: Microservices Architecture

15.1 Microservices vs Monolith

Aspect	Monolith	Microservices
Deployment	Single unit	Independent services
Scaling	Scale entire app	Scale individual services
Technology	Single stack	Polyglot
Complexity	Simpler initially	Higher operational complexity
Team Structure	One large team	Small autonomous teams

15.2 Service Communication

- Synchronous: REST, gRPC (request-response)
- Asynchronous: Message queues (RabbitMQ, Kafka)

15.3 Service Discovery

Services need to find each other dynamically.

- Client-side: Client queries service registry
- Server-side: Load balancer queries registry

Chapter 16: System Design Case Studies

16.1 Design a URL Shortener (like bit.ly)

Requirements:

- Shorten long URLs to short aliases
- Redirect short URLs to original
- Custom short links
- Analytics (optional)

High-Level Design:

```
[Client] -> [Load Balancer] -> [API Servers]
                |
            [Cache (Redis)]
                |
            [Database]
```

Key Considerations:

- URL Encoding: Base62 encoding of auto-increment ID
- Database: Key-value store (short URL -> long URL)
- Cache: Redis for hot URLs
- Rate Limiting: Prevent abuse

16.2 Design a Twitter Feed

Requirements:

- Post tweets
- Follow/unfollow users

- View news feed (tweets from followed users)

Feed Generation Approaches:

Approach	Description	Pros/Cons
Fan-out on Write	Push tweets to followers' feeds	Fast read, slow write
Fan-out on Read	Generate feed on request	Slow read, fast write
Hybrid	Push to active users, pull for others	Balanced approach

16.3 Design a Chat Application

Key Components:

- WebSocket servers for real-time communication
- Message store (database)
- Presence service (who's online)
- Push notification service

16.4 More Practice Problems

System	Key Concepts
Design YouTube	CDN, video encoding, metadata storage
Design Uber	GPS tracking, ETA calculation, matching
Design WhatsApp	Message queue, end-to-end encryption
Design Rate Limiter	Token bucket, sliding window
Design Key-Value Store	Consistent hashing, replication

Part 4: Android Development Interviews

Android development interviews test your knowledge of the Android platform, Kotlin programming, architecture patterns, and best practices. This part covers everything you need to ace Android interviews.

Chapter 17: Android Fundamentals

17.1 Android Architecture

Android is built on a layered architecture:

Layer	Components
Applications	System apps and user apps
Application Framework	Activity Manager, Window Manager, Content Providers
Libraries	SQLite, WebKit, Media Framework
Android Runtime	ART, Core Libraries
Hardware Abstraction	HAL for device drivers
Linux Kernel	Memory, process, driver management

17.2 Core Components

Four Main Components:

- Activities: Single screen with UI
- Services: Background operations without UI
- Broadcast Receivers: Respond to system-wide events
- Content Providers: Manage shared app data

17.3 Activity Lifecycle

```
// Activity Lifecycle Callbacks
onCreate()      -> Activity is created
```

```
onStart()       -> Activity becomes visible
onResume()      -> Activity gains focus
onPause()       -> Activity loses focus
onStop()        -> Activity no longer visible
onRestart()     -> Activity restarts after
stop onDestroy()                    ->
Activity is destroyed
```

17.4 Intent and Intent Filters

Intents are messaging objects used to request actions from other components.

```
// Explicit Intent
val intent = Intent(this, SecondActivity::class.java)
intent.putExtra("key", "value")
startActivity(intent)

// Implicit Intent
val intent = Intent(Intent.ACTION_VIEW)
intent.data =
Uri.parse("https://www.example.com")
startActivity(intent)
```

Chapter 18: Kotlin for Android

18.1 Why Kotlin?

Kotlin is now the preferred language for Android development. It offers:

- Concise syntax - less boilerplate code
- Null safety - prevents null pointer exceptions
- Extension functions - add functionality to existing classes
- Coroutines - simplified async programming
- Interoperability - works with Java

18.2 Kotlin Essentials

```
// Variables
val immutable = "constant"  // Cannot be
reassigned var mutable = "changeable" // Can be
reassigned

// Null Safety
var nonNull: String = "value"
var nullable: String? = null
val length = nullable?.length ?: 0  // Elvis operator

// Extension Function
fun String.addExclamation(): String = this + "!"
val excited = "Hello".addExclamation()

// Data Class
data class User(val name: String, val age: Int)
val user = User("John", 25)
val (name, age) = user  // Destructuring

// When Expression (enhanced switch)
when (age) {
   in 0..12 -> "Child"
   in 13..19 -> "Teenager"
   else -> "Adult"
}
```

18.3 Kotlin Coroutines

Coroutines simplify asynchronous programming.

```
// Basic Coroutine
lifecycleScope.launch {
```

```
    val result = fetchData()  // Suspend function
    updateUI(result)
}

// WithContext for thread switching
suspend fun fetchData(): String = withContext(Dispatchers.IO) {
    // Network call on IO thread
    api.getData()
}

// async/await for parallel execution
val deferred1 = async { fetchUser() }
val deferred2 = async { fetchOrders()
} val user = deferred1.await()
val orders = deferred2.await()

// Flow for streams of data
viewModel.uiState
    .flowWithLifecycle(lifecycle)
    .collect { state ->
        updateUI(state)
    }
```

Chapter 19: Android Architecture Components

19.1 MVVM Architecture

Model-View-ViewModel is the recommended architecture pattern.

Component	Responsibility
Model	Data layer (Repository, Database, API)

View	UI layer (Activity, Fragment) - observes ViewModel
ViewModel	Holds UI data, survives config changes

```
// ViewModel
class UserViewModel(private val repository: UserRepository) : ViewModel() {

    private val _users = MutableLiveData<List<User>>()
    val users: LiveData<List<User>> = _users

    fun loadUsers() {
        viewModelScope.launch {
            _users.value = repository.getUsers()
        }
    }
}

// Fragment
class UserFragment : Fragment() {
    private val viewModel: UserViewModel by viewModels()

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        viewModel.users.observe(viewLifecycleOwner) { users ->
            adapter.submitList(users)
        }
    }
}
```

19.2 Room Database

Room is an abstraction layer over SQLite.

```
// Entity
```

```
@Entity(tableName = "users")
data class User(
    @PrimaryKey val id: Int,
    @ColumnInfo(name = "first_name") val firstName: String,
    @ColumnInfo(name = "last_name") val lastName: String
)

// DAO
@Dao
interface UserDao {
    @Query("SELECT * FROM users")
    fun getAll(): Flow<List<User>>

    @Insert
    suspend fun insert(user: User)

    @Delete
    suspend fun delete(user: User)
}

// Database
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

19.3 Dependency Injection with Hilt

```
// Application class
@HiltAndroidApp
class MyApplication : Application()
```

```
// Module
@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides
    @Singleton
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
    }
}

// Inject in ViewModel
@HiltViewModel
class UserViewModel @Inject constructor(
    private val repository: UserRepository
) : ViewModel()
```

Chapter 20: Android Performance & Optimization

20.1 Memory Management

- Avoid memory leaks - use WeakReference for callbacks
- Use ViewBinding instead of findViewById
- Recycle views with RecyclerView
- Use LeakCanary to detect memory leaks

20.2 UI Performance

- Minimize overdraw - avoid unnecessary backgrounds
- Use ConstraintLayout for flat hierarchies
- Offload work from main thread
- Use RecyclerView with DiffUtil

20.3 Network Optimization

- Use OkHttp with caching
- Implement pagination
- Compress images
- Use WorkManager for background sync

Chapter 21: Android Interview Questions

Q: What is the difference between Activity and Fragment?

A:Activity is a single, focused screen with UI. Fragment is a reusable UI component that can be embedded in Activities. Fragments have their own lifecycle and can be combined in a single Activity to build multi-pane UIs. Fragments were introduced to support more flexible UI designs for different screen sizes.

Q: Explain the differences between Service, IntentService, and JobIntentService.

A:Service runs on the main thread by default and can run indefinitely. IntentService runs on a separate worker thread and stops itself when work is done. JobIntentService is the modern replacement for IntentService that works with Android O+ background restrictions using JobScheduler.

Q: What is the difference between LiveData and StateFlow?

A:LiveData is lifecycle-aware and automatically handles subscription based on lifecycle state. StateFlow is a Kotlin Flow API that always has a value and requires manual lifecycle handling (usually with repeatOnLifecycle). StateFlow is more flexible for complex reactive scenarios.

Q: How do you handle configuration changes in Android?

A:Use ViewModel to survive configuration changes. Store UI-related data in ViewModel. For complex cases, use onSaveInstanceState for small amounts of data. Avoid using configChanges attribute in manifest as a quick fix.

Q: What are the differences between Serializable and Parcelable?

A:Serializable is a Java interface that uses reflection and is slow. Parcelable is Android-specific, requires manual implementation, but is much faster and recommended for Android. Use Parcelable for passing data between components.

Q: Explain the Android app launch process.

A:1) User taps app icon. 2) System creates app process and Application instance. 3) Main thread (UI thread) starts. 4) Application.onCreate() called. 5) MainActivity instance created. 6) Activity lifecycle methods called (onCreate, onStart, onResume). 7) Content view inflated and displayed.

Part 5: Web Development Interviews

Web development interviews cover frontend, backend, and full-stack skills. This part prepares you for questions on HTML, CSS, JavaScript, React, Node.js, and more.

Chapter 22: Frontend Fundamentals

22.1 HTML5 Essentials

```
<!-- Semantic HTML -->
<header>
  <nav>
    <ul>
      <li><a href="/">Home</a></li>
    </ul>
  </nav>
</header>

<main>
  <article>
    <h1>Article Title</h1>
    <section>Content</section>
  </article>

  <aside>Related content</aside>
</main>

<footer>Copyright info</footer>
```

22.2 CSS Fundamentals

```
/* Flexbox */
```

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 1rem;
}

/* Grid */
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

/* Responsive Design */
@media (max-width: 768px) {
  .grid {
    grid-template-columns: 1fr;
  }
}

/* CSS Variables */
:root {
  --primary-color: #007bff;
}
.button {
  background-color: var(--primary-color);
}
```

22.3 CSS Specificity

Specificity determines which CSS rule applies:

Selector	Specificity
Inline styles	1000
ID (#id)	100
Class (.class), Attribute, Pseudo-class	10
Element, Pseudo-element	1

Chapter 23: JavaScript Deep Dive

23.1 JavaScript Fundamentals

```
// Variable declarations
const constant = 'cannot
reassign'; let mutable = 'can
reassign';
// Avoid var - function scoped, can be redeclared

// Arrow Functions
const add = (a, b) => a + b;

// Destructuring
const { name, age } = person;
const [first, second] = array;

// Spread Operator
const newArray = [...oldArray, newItem];
const newObj = { ...oldObj, newProp: value };

// Template Literals
const message = `Hello, ${name}!`;
```

```
// Promises
fetch('/api/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));

// Async/Await
async function getData() {
  try {
    const response = await fetch('/api/data');
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(error);
  }
}
```

23.2 Closures

A closure is a function that has access to variables in its outer scope even after the outer function returns.

```
function createCounter() {
  let count = 0;

  return {
    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count
  };
}
```

```
const counter = createCounter();
console.log(counter.increment()); // 1
console.log(counter.increment()); // 2
console.log(counter.getCount());  // 2
```

23.3 Prototype and Inheritance

```
// Prototype Chain
function Person(name) {
this.name = name;
}

Person.prototype.greet = function() {
  return `Hello, I'm ${this.name}`;
};

// ES6 Classes (syntactic sugar over prototypes)
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    return `Hello, I'm ${this.name}`;
  }
}

class Student extends Person {
  constructor(name, grade) {
  super(name);
```

```
    this.grade = grade;
  }
}
```

23.4 Event Loop

JavaScript is single-threaded but handles async operations via the event loop.

```
// Execution order example
console.log('1'); // Synchronous

setTimeout(() => console.log('2'), 0); // Macrotask

Promise.resolve().then(() => console.log('3')); // Microtask

console.log('4'); // Synchronous

// Output: 1, 4, 3, 2
// Microtasks execute before next macrotask
```

Chapter 24: React & Modern Frameworks

24.1 React Fundamentals

```
// Functional Component with Hooks
import { useState, useEffect } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
```

```
    return () => {
      // Cleanup function
      document.title = 'React App';
    };
  }, [count]);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

24.2 React Hooks

Hook	Purpose
useState	Manage state in functional components
useEffect	Side effects (data fetching, subscriptions)
useContext	Access React context
useRef	Mutable references, DOM access
useMemo	Memoize expensive computations
useCallback	Memoize callbacks
useReducer	Complex state logic

24.3 React Performance Optimization

```
// React.memo - prevent unnecessary re-renders
```

```
const MyComponent = React.memo(function MyComponent({ name }) {
  return <div>{name}</div>;
});

// useMemo - memoize computed values const expensiveValue = useMemo(() => { return computeExpensiveValue(a, b);
}, [a, b]);

// useCallback - memoize callbacks
const handleClick = useCallback(() => {
doSomething(a, b);
}, [a, b]);

// Code Splitting
const LazyComponent = lazy(() => import('./LazyComponent'));
```

24.4 State Management

Solution	Best For
useState/useReducer	Local component state
Context API	Prop drilling, theme/auth
Redux	Large apps, complex state
Zustand	Simple global state
React Query	Server state management

Chapter 25: Backend Development

25.1 REST API Design

```
// RESTful Endpoints
```

```
GET    /api/users        // List all users
GET                      // Get specific user
                         // Create new user
/api/users/:id           // Update user (full)
PATCH                    // Update user
                         (partial)
DELETE /api/users/:id    // Delete
```

25.2 Node.js & Express

```
const express = require('express');
const app = express();

// Middleware
app.use(express.json());

// Routes
app.get('/api/users', async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.post('/api/users', async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).json(user);
  } catch (error) {
```

```
    res.status(400).json({ error: error.message });
  }
});

// Error handling middleware
app.use((err, req, res, next) => {
console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!' });
});
```

25.3 Database Concepts

```
// SQL Example
SELECT u.name, COUNT(o.id) as order_count
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE u.created_at > '2023-01-01'
GROUP BY u.id
HAVING order_count > 5
ORDER BY order_count DESC
LIMIT 10;

// Indexing for performance
CREATE INDEX idx_user_email ON users(email);
```

Chapter 26: Full Stack Interview Questions

Q: What is CORS and how do you handle it?

A:CORS (Cross-Origin Resource Sharing) is a security feature that restricts web pages from making requests to a different domain. Handle it by: 1) Configuring proper CORS headers on the server (Access-Control-Allow-Origin), 2) Using a proxy server in development, 3) Implementing JSONP for

simple GET requests (legacy), 4) Using same-origin policy for production.

Q: Explain the Virtual DOM in React.

A:Virtual DOM is a lightweight JavaScript representation of the actual DOM. When state changes, React creates a new Virtual DOM, compares it with the previous one (diffing), and only updates the changed parts in the real DOM. This minimizes expensive DOM operations and improves performance.

Q: What are the differences between cookies, localStorage, and sessionStorage?

A:Cookies: Small size (4KB), sent with every HTTP request, can have expiration. localStorage: Larger (5-10MB), persists until cleared, not sent with requests. sessionStorage: Same as localStorage but cleared when tab closes. Use cookies for auth tokens, localStorage for user preferences, sessionStorage for temporary data.

Q: How does JWT authentication work?

A:1) User logs in with credentials. 2) Server validates and creates JWT (header.payload.signature). 3) Client stores JWT (usually in memory or httpOnly cookie). 4) Client sends JWT in Authorization header for protected requests. 5) Server verifies signature and extracts user info from payload. 6) Access token is short-lived, refresh token gets new access tokens.

Q: Explain server-side rendering (SSR) vs client-side rendering (CSR).

A:SSR: Server renders HTML on each request. Pros: Better SEO, faster initial load. Cons: More server load, full page reloads. CSR: Browser renders content using JavaScript. Pros: Rich interactions, less server load. Cons: Poor SEO without extra work, slower initial load. Modern approach: Use frameworks like Next.js for hybrid rendering.

Part 6: Behavioral & HR Interviews

Behavioral interviews assess your soft skills, cultural fit, and how you handle real-world situations. This part teaches you how to structure compelling answers and make a great impression.

Chapter 27: Understanding Behavioral Interviews

27.1 Why Behavioral Interviews Matter

Companies use behavioral interviews to:

- Assess cultural fit and values alignment
- Understand your problem-solving approach
- Evaluate communication and teamwork skills
- Predict future behavior based on past actions

27.2 What Interviewers Look For

Quality	What They Want to See
Leadership	Taking initiative, influencing others
Teamwork	Collaboration, conflict resolution
Problem-Solving	Analytical thinking, creativity
Adaptability	Handling change, learning from failure
Communication	Clear expression, active listening
Results-Orientation	Focus on outcomes, impact

Chapter 28: STAR Method Mastery

28.1 What is the STAR Method?

STAR is a structured approach for answering behavioral questions:

Letter	Meaning	What to Include

S	Situation	Set the context - where, when, what
T	Task	Your responsibility or challenge
A	Action	What YOU did (use 'I', not 'we')
R	Result	Outcome, quantified if possible

28.2 STAR Example

Question: Tell me about a time you had to meet a tight deadline.

SITUATION:
"In my previous role, our team was tasked with delivering a mobile app feature for a major client. Two weeks before the deadline, our lead developer had to take unexpected medical leave."

TASK:
"I was asked to take over the critical payment integration module, which was about 60% complete but had several blocking bugs."

ACTION:
"I immediately reviewed the existing code and identified the three main issues. I created a detailed plan with daily milestones and communicated this to the project manager. I worked closely with the QA team, setting up daily check-ins to catch issues early. I also documented my progress daily so others could pick up if needed."

RESULT:
"We delivered the feature on time with zero critical bugs. The client was so satisfied they increased their contract with us by 40%. I also created documentation that helped us reduce similar integration time by 30% on future projects."

28.3 Common Mistakes to Avoid

- Being too vague - use specific examples
- Using 'we' instead of 'I' - focus on your contribution
- Skipping the result - always share the outcome
- Talking too long - keep it 2-3 minutes
- Not preparing - have 8-10 stories ready

Chapter 29: Common Behavioral Questions

29.1 Leadership & Initiative

Q: Tell me about a time you took initiative.

A:Focus on identifying a problem or opportunity without being asked, taking action, and creating positive impact. Example: Improving a process, learning a new skill, or starting a project.

Q: Describe a situation where you had to influence someone without authority.

A:Show how you used data, built relationships, found common ground, or demonstrated value to persuade others.

29.2 Teamwork & Collaboration

Q: Tell me about a conflict you had with a teammate.

A:Focus on: 1) The situation without blame, 2) How you listened and understood their perspective, 3) Finding a compromise or solution, 4) What you learned. Never speak negatively about others.

Q: Describe a time you worked with a difficult colleague.

A:Emphasize empathy, communication, and focusing on shared goals. Show you can work with diverse personalities.

29.3 Problem-Solving & Challenges

Q: Tell me about a time you failed.

A:Choose a real failure (not trivial), explain what happened, what you learned, and how you applied that lesson. Show growth mindset.

Q: Describe a complex problem you solved.

A:Walk through your thought process: how you broke it down, what approaches you considered, and why you chose your solution.

29.4 Adaptability & Growth

Q: Tell me about a time you had to learn something quickly.

A:Show your learning process: resources used, practice methods, and how you applied the new skill.

Q: How do you handle feedback?

A:Emphasize that you welcome feedback, listen actively, ask clarifying questions, and take action to improve.

Chapter 30: HR Interview Strategies

30.1 Common HR Questions

Q: Tell me about yourself.

A:Structure: 1) Current role and relevant experience, 2) Key achievements, 3) Why you're interested in this role. Keep it 1-2 minutes. Example: 'I'm a software engineer with 3 years of experience building Android apps. At my current company, I led the development of a feature that increased user engagement by 40%. I'm passionate about creating great user experiences and I'm excited about this role because...'

Q: Why do you want to work here?

A:Show you've researched the company. Mention: company mission, products/services you admire, culture, growth opportunities, or specific projects. Be genuine and specific.

Q: What are your salary expectations?

A:Research market rates beforehand. Give a range based on your research, experience, and location. Example: 'Based on my research of similar roles in this area and my experience level, I'm looking for a

range of X to Y. I'm open to discussion based on the total compensation package.'

Q: Where do you see yourself in 5 years?

A:Show ambition while being realistic. Focus on skills you want to develop, impact you want to make, and growth within the company. Avoid answers that suggest you'll leave soon.

30.2 Questions to Ask the Interviewer

- What does success look like in this role in the first 6 months?
- What are the biggest challenges the team is facing?
- How would you describe the team culture?
- What opportunities for growth and learning are available?
- What is the typical career path for someone in this role?
- What are the next steps in the interview process?

Part 7: Resume & Portfolio Building

Your resume and portfolio are your first impression. This part teaches you how to create documents that get you noticed and land interviews.

Chapter 31: Creating an ATS-Friendly Resume

31.1 Understanding ATS

Applicant Tracking Systems (ATS) are software that scans resumes before a human sees them. 75% of resumes are rejected by ATS before reaching a recruiter.

31.2 Resume Format Best Practices

- Use standard fonts: Arial, Calibri, Times New Roman
- Avoid tables, columns, headers, footers, and graphics
- Save as .docx or .pdf (check job posting)
- Use standard section headings: Experience, Education, Skills
- Keep it 1 page (junior) or 2 pages (experienced)

31.3 Resume Structure

```
[NAME]
[Email] | [Phone] | [LinkedIn] | [GitHub/Portfolio]

SUMMARY (2-3 lines)
Software Engineer with 3 years of experience building scalable
web applications. Proficient in React, Node.js, and cloud
deployment. Passionate about clean code and user experience.

TECHNICAL SKILLS
Languages: JavaScript, Python, Java
Frontend: React, HTML5, CSS3, TypeScript
Backend: Node.js, Express, REST APIs
```

Database: MongoDB, PostgreSQL, Redis
Cloud: AWS (EC2, S3), Docker, Kubernetes
Tools: Git, Jenkins, Jest

EXPERIENCE
Company Name | Software Engineer | Jan 2022 - Present
- Developed responsive web app using React and TypeScript, improving page load time by 40%
- Implemented CI/CD pipeline with GitHub Actions, reducing deployment time from 2 hours to 15 minutes
- Led migration from REST to GraphQL, reducing API calls by 60%
- Mentored 2 junior developers through code reviews and pair programming

Previous Company | Junior Developer | Jun 2020 - Dec 2021
- Built RESTful APIs serving 100K+ daily requests
- Optimized database queries, reducing response time by 50%

PROJECTS
Project Name | [GitHub Link]
- Built full-stack e-commerce platform with React and Node.js
- Implemented payment processing with Stripe API
- Deployed on AWS with auto-scaling configuration

EDUCATION
University Name | Bachelor of Science in Computer Science | 2020

31.4 Action Verbs & Metrics

Strong Action Verbs:

- Developed, Built, Implemented, Designed

- Optimized, Improved, Reduced, Increased
- Led, Managed, Mentored, Collaborated

Always Include Metrics:

- Improved performance by X%
- Reduced load time from X to Y seconds
- Served X users/day
- Increased conversion by X%

Chapter 32: Building Your Portfolio

32.1 Why You Need a Portfolio

- Shows you can build real things
- Demonstrates your skills better than words
- Gives interviewers something to discuss
- Sets you apart from other candidates

32.2 Project Ideas by Domain

Domain	Project Ideas
Android	Todo app, Weather app, E-commerce app, Chat app
Web Frontend	Portfolio site, Dashboard, Social media clone
Full Stack	Blog platform, E-commerce site, Booking system
Backend	API service, Authentication system, URL shortener
DevOps	CI/CD pipeline, Infrastructure as Code, Monitoring setup

32.3 What Makes a Good Portfolio Project

- Solves a real problem or demonstrates key skills
- Has clean, well-documented code

- Is deployed and accessible
- Includes a README with setup instructions
- Has tests (unit, integration)
- Shows your thought process (blog post or video)

Chapter 33: GitHub & Online Presence

33.1 Optimizing Your GitHub Profile

- Professional profile picture
- Clear bio with your expertise
- Pin your best 6 repositories
- Contribute to open source
- Maintain green contribution graph
- Write good commit messages

33.2 LinkedIn Optimization

- Professional headshot
- Compelling headline (not just job title)
- Detailed experience with metrics
- Skills endorsements and recommendations
- Regular posts about your learning
- Engage with others' content

Chapter 34: Cover Letters & Applications

34.1 When to Write a Cover Letter

- Required by job posting
- Changing career paths

- Explaining employment gaps
- Applying to competitive companies

34.2 Cover Letter Structure

[Your Name]
[Your Contact Information]

[Date]

[Hiring Manager Name or 'Hiring Manager']
[Company Name]

Dear [Name],

PARAGRAPH 1: Hook
Mention the role you're applying for and why you're excited about it.
Reference a specific product, value, or achievement of the company.

PARAGRAPH 2: Why You're Qualified
Highlight 2-3 key qualifications that match the job requirements.
Use specific examples with metrics.

PARAGRAPH 3: Why This Company
Show you've done your research.
Connect your values/career goals to the company.

PARAGRAPH 4: Call to Action
Express enthusiasm for an
interview. Thank them for their
time.

Sincerely,

[Your Name]

Part 8: Company-Specific Preparation

Different companies have different interview styles and expectations. This part helps you prepare for specific types of companies.

Chapter 35: FAANG Companies

35.1 Google Interview Process

Round	Focus
Recruiter Screen	Basic fit, resume review
Phone Screen	1-2 coding problems
Onsite/Virtual	4-5 rounds: coding, system design, behavioral, Googliness

Google-Specific Tips:

- Focus on coding - very high bar for algorithms
- System design for L4+ positions
- Googliness: collaboration, intellectual humility
- Practice on Google Code Jam problems

35.2 Amazon Interview Process

Round	Focus
OA	2 coding problems + work simulation
Phone Screen	Coding + LP questions
Onsite/Virtual	4 rounds: coding, system design, behavioral (LP heavy)

Leadership Principles - Prepare 2 stories for each:

- Customer Obsession
- Ownership

- Invent and Simplify
- Are Right, A Lot
- Learn and Be Curious
- Hire and Develop the Best
- Insist on the Highest Standards
- Think Big
- Bias for Action
- Frugality
- Earn Trust
- Dive Deep
- Have Backbone; Disagree and Commit
- Deliver Results
- Strive to be Earth's Best Employer
- Success and Scale Bring Broad Responsibility

35.3 Meta/Facebook Interview Process

Round	Focus
Recruiter Screen	Basic fit
Phone Screen	Coding
Onsite/Virtual	Coding, System Design, Behavioral

Meta-Specific Tips:

- Move fast culture - show you can iterate quickly
- Boldness - take on ambitious challenges
- Focus on impact - everything ties back to impact

35.4 Apple Interview Process

Apple-Specific Tips:

- Strong focus on product sense and user experience
- Secrecy culture - show you can be trusted
- Attention to detail matters
- Passion for Apple products is a plus

Chapter 36: Startups vs Big Tech

36.1 Startup Interviews

Aspect	Startup
Process	Often faster, less structured
Focus	Breadth over depth, culture fit
Questions	May include take-home projects
What They Want	Self-starters, generalists

Startup Interview Tips:

- Show you can wear multiple hats
- Ask about funding and runway
- Demonstrate ownership mentality
- Be prepared for equity discussions

36.2 Big Tech Interviews

Aspect	Big Tech
Process	Structured, multiple rounds
Focus	Depth in specific areas
Questions	Standardized, algorithm-heavy
What They Want	Specialists, scalable thinking

Chapter 37: Service-Based Companies

37.1 Common Service-Based Companies

- TCS, Infosys, Wipro, Cognizant
- Accenture, Capgemini, IBM
- HCL, Tech Mahindra, LTI

37.2 Interview Pattern

Round	Focus
Aptitude Test	Quant, Logical, Verbal
Technical Test	Basic programming, SQL, pseudocode
Technical Interview	Projects, basic concepts, coding
HR Interview	Communication, willingness to relocate

37.3 Preparation Strategy

- Strong aptitude practice (RS Aggarwal)
- Know your projects thoroughly
- Basic OOPs, DBMS, CN concepts
- Simple coding problems
- Willingness to learn and adapt

Part 9: Mock Interviews & Practice

Practice is the key to interview success. This part provides practice problems, mock scenarios, and a final checklist to ensure you're fully prepared.

Chapter 38: Practice Problems by Topic

38.1 Arrays & Strings - Must Do

Problem	Difficulty	Why Important
Two Sum	Easy	Hash map basics
Best Time to Buy/Sell Stock	Easy	One-pass algorithm
Contains Duplicate	Easy	Hash set usage
Product of Array Except Self	Medium	Prefix/suffix technique
Maximum Subarray	Medium	Kadane's algorithm
3Sum	Medium	Two pointers
Merge Intervals	Medium	Sorting + intervals
Trapping Rain Water	Hard	Two pointers, stack

38.2 Linked Lists - Must Do

Problem	Difficulty
Reverse Linked List	Easy
Merge Two Sorted Lists	Easy
Linked List Cycle	Easy
Remove Nth Node from End	Medium
Add Two Numbers	Medium
Copy List with Random Pointer	Medium
Merge K Sorted Lists	Hard

38.3 Trees & Graphs - Must Do

Problem	Difficulty
Maximum Depth of Binary Tree	Easy
Same Tree	Easy
Invert Binary Tree	Easy
Binary Tree Level Order	Medium
Validate BST	Medium
Lowest Common Ancestor	Medium
Number of Islands	Medium
Course Schedule	Medium

38.4 Dynamic Programming - Must Do

Problem	Difficulty
Climbing Stairs	Easy
House Robber	Easy
Coin Change	Medium
Longest Increasing Subsequence	Medium
Word Break	Medium
Edit Distance	Medium
Regular Expression Matching	Hard

38.5 System Design - Must Practice

System	Key Concepts to Cover
URL Shortener	Hashing, database, cache
Twitter Feed	Fan-out, timeline generation
Chat Application	WebSockets, message queue

YouTube	CDN, video encoding, metadata
Uber	GPS, matching algorithm, ETA
Rate Limiter	Token bucket, sliding window
Key-Value Store	Consistent hashing, replication

Chapter 39: Mock Interview Scenarios

39.1 Coding Interview Simulation

Scenario 1: Two Sum

Problem: Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

Sample Answer Flow:

```
1. UNDERSTAND
   - Can the same element be used twice? No.
   - Is there always exactly one solution? Yes.
   - What if no solution exists? Return empty/null.

2. BRUTE FORCE APPROACH
   - Nested loops: O(n^2) time, O(1) space
   - Check every pair

3. OPTIMIZED APPROACH
   - Use hash map to store complement
   - O(n) time, O(n) space
   - Single pass through array

4. CODE
   public int[] twoSum(int[] nums, int target)
      { Map<Integer, Integer> map = new
      HashMap<>();
```

```
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];

        if (map.containsKey(complement)) {
            return new int[] { map.get(complement), i };
        }

        map.put(nums[i], i);
    }

    return new int[] {};
}
```

```
5. TEST
   Input: [2,7,11,15], target = 9
   - i=0: complement=7, map empty, put(2,0)
   - i=1: complement=2, found! return [0,1]

6. COMPLEXITY
   Time: O(n) - single pass
   Space: O(n) - hash map storage
```

39.2 System Design Simulation

Scenario: Design a URL Shortener

Sample Answer Flow:

```
1. REQUIREMENTS
   Functional:
   - Shorten long URL to short alias
   - Redirect short URL to original
   - Custom short links (optional)
```

- Link expiration (optional)

Non-Functional:
- High availability
- Low latency redirects
- Scale: 100M URLs/day

2. ESTIMATION
- 100M new URLs/day = ~1160 URLs/second
- Read:Write ratio ~100:1
- URL size: ~500 bytes original, ~50 bytes short
- 5-year storage: ~100M * 365 * 5 * 500B = ~91TB

3. HIGH-LEVEL DESIGN

```
Client -> Load Balancer -> API Servers -> Cache -> Database
                    |
                Encoding Service
```

4. DEEP DIVE

Encoding:
- Base62 encoding of auto-increment ID
- 7 characters = 62^7 = ~3.5 trillion URLs

Database:
- NoSQL (Cassandra/DynamoDB) for horizontal scaling
- Schema: short_url (PK), long_url, created_at, expires_at

Cache:
- Redis for hot URLs (80-20 rule)

- TTL based on access patterns

Scaling:
- Read replicas for redirect traffic
- Sharding by short_url prefix

5. TRADE-OFFS
- Base62 vs Hash: Hash can collide, Base62 is sequential
- SQL vs NoSQL: NoSQL better for write-heavy, scale needs
- Cache write-through vs write-back: Consistency vs speed

39.3 Behavioral Interview Simulation

Question: Tell me about a time you had a conflict with a teammate.

Sample Answer (STAR Method):

```
SITUATION:
"In my previous role, I was working on a feature with a teammate
who had a very different coding style than mine. We disagreed on
whether to use a third-party library or build the functionality
ourselves.
He wanted to build from scratch; I preferred using a well-tested library."

TASK:
"We needed to resolve this quickly as it was blocking our sprint. I
needed to find a solution that satisfied both of our concerns."

ACTION:
"I suggested we take 30 minutes to prototype both approaches. I built
a quick proof-of-concept with the library while he built a minimal
version from scratch. We then evaluated both on:
- Development time
- Performance
```

- Maintenance burden
- Security

After comparing, the library approach was clearly better for our timeline and requirements. He appreciated that we actually tested both approaches rather than just arguing."

RESULT:
"We completed the feature 2 days ahead of schedule. My teammate later told me he appreciated my approach of using data rather than opinions. We established a pattern for future disagreements: always prototype and measure before deciding."

Chapter 40: Final Preparation Checklist

40.1 Technical Preparation

Task	Status
Solved 100+ LeetCode problems	[ ]
Covered all major data structures	[ ]
Practiced system design cases	[ ]
Reviewed domain-specific topics	[ ]
Timed practice sessions	[ ]
Mock interviews with peers	[ ]

40.2 Behavioral Preparation

Task	Status
Prepared 8-10 STAR stories	[ ]
Practiced answers out loud	[ ]

Researched company thoroughly	[ ]
Prepared questions to ask	[ ]
Mock behavioral interview	[ ]

40.3 Resume & Portfolio

Task	Status
Resume updated and proofread	[ ]
Portfolio projects deployed	[ ]
GitHub profile optimized	[ ]
LinkedIn profile updated	[ ]
References ready	[ ]

40.4 Day-Before Checklist

Task	Status
Tested tech setup (camera, mic, internet)	[ ]
Prepared outfit	[ ]
Planned route / quiet space	[ ]
Printed resume (if in-person)	[ ]
Good night's sleep	[ ]
Light review only	[ ]

40.5 Final Tips for Success

- Stay calm - interviews are conversations, not interrogations
- Think out loud - share your thought process
- Ask clarifying questions - do not assume
- Test your code - run through examples
- Be honest - it is okay to not know everything

- Show enthusiasm - companies want people who want to be there
- Learn from each interview - improve for the next one
- Do not give up - persistence pays off

Good Luck with Your Interviews!

Remember: Preparation + Practice + Persistence = Success

Interview Preparation

Complete A-Z Guide for All Tech Roles

Created & Authored by

Mohammad Faiz

Year 2026

Copyright (c) 2026 Mohammad Faiz. All Rights Reserved.

No Permission to Use - Personal Use Only - Attribution Required